Amazon Appliances Data Cleaning

Zitong Guo | DSO 576 Module 6 | Individual homework

This submission cleans and verifies all 1,000 reviews in the team-supplied project file. It does not cover the full upstream Appliances category. All source rows and values are preserved; 314 review bodies and 13 titles are normalized in additional columns.

Submission and source

Personal branch: Jenny-Guo

Branch URL: https://github.com/dtry19/dso576-project-/tree/Jenny-Guo

Submitted code/results commit: d816d115a2805ef63d5d023ef74e4a77dc7c2949

The report and commit reference are added in a subsequent documentation commit; the identifier above fixes the tested cleaning implementation and outputs.

Source file: dataset/Appliances_sample.jsonl
Team repository source commit: 2a623b24ff6e1a427426707117e4e45ed4d948c2
Upstream: McAuley Lab Amazon Reviews 2023
https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023

Source SHA-256: 6e5ce91e627c1eb52471166a207b8b76b36e062835a544bcd793af4d743f1608

The source sampling procedure and original download date are unknown. The committed source version and checksum make this input reproducible. Product metadata is absent; the required merge validation remains unresolved.

Verified results

Measure	Before	After
Review rows	1,000	1,000
Rating sum / valid count	4,406 / 1,000	4,406 / 1,000
Average rating	4.406	4.406
Ratings 1 or 2 / valid count	105 / 1,000 (10.5%)	105 / 1,000 (10.5%)
Helpful votes / valid count	1,229 / 1,000	1,229 / 1,000

Checklist items 1 through 7

1. Project vision and question - Done

An Amazon Appliances category manager must prioritize listings for product-information review and attributes for standardization within comparable appliance types. Question: which listings warrant review for recurring customer requirements potentially missing or unclear in their listings? This cleaning stage provides review counts, average ratings, low-rating shares and helpful-vote totals; a mismatch rate is not yet measurable. Evidence: cleaning_rules.md and output/product_metrics.json.

2. Define one row - Done

One row is one review. source_row uniquely identifies a row within this exact file. The candidate (user_id, asin, timestamp) combination has zero collisions here, but is not assumed universally unique. parent_asin identifies a parent listing and can repeat. Evidence: output/audit.json.

3. Preserve the source - Done

Read every JSONL line from the committed source file; malformed JSON stops the run. SHA-256 is unchanged after cleaning. Original fields remain alongside cleaned fields, and row-by-row raw-field equality is asserted. Source version and loading instructions are in README.md.

4. Inspect columns and types - Done

All 1,000 ratings are valid integral floats, converted into integer rating_clean. All timestamps are integer Unix milliseconds, converted to timestamp_utc; originals are retained. IDs/title/text are strings, images lists, votes integers, and purchase flags Booleans. Zero failures for ratings, dates, votes or purchase flags. Evidence: audit.source_types and conversion_failures.

5. Standardize text - Done

Replace known HTML break tags with spaces, decode HTML entities once and collapse whitespace. Preserve case, punctuation, negation and model identifiers. 314 bodies and 13 titles change. Row 4: "filters.

I can" becomes "filters. I can". Other literal tags are preserved. IDs remain original strings, including any leading zeros. Evidence: clean.py and record check 2.

6. Investigate duplicates - Done

Zero exact duplicate rows; zero user/variant/timestamp collisions. 829 distinct parent IDs: 109 repeat, with 171 excess occurrences. 870 variant IDs: 90 repeat, with 130 excess occurrences. 625 user IDs: 187 repeat. Keep all nonidentical reviews. Only exact duplicates would be removed. Evidence: audit.repeated_ids and excluded_rows.json (empty).

7. Handle missing data - Done

Each of the 10 source columns has 0 missing values before and after; all 6 added cleaned value fields also have 0 missing values. No imputation or missing-value deletion is needed. The 957 empty image lists mean no attached images, not missing reviews. Full per-column counts are in audit.missing_before and missing_after.

Checklist items 8 through 14

8. Check suspicious values - Done

All ratings are integers from 1 to 5; helpful votes are nonnegative integers; purchase flags are Boolean; ASIN formats are valid. Dates range from 2009-09-27 to 2023-03-14 UTC, within the documented source window. No quality flags occur. Retain maximum helpful count 185, all 102 unverified purchases and 389 variant/parent differences. These are not automatically errors. Evidence: audit.json.

9. Verify merges - Unresolved

No metadata was supplied and no merge was performed. Future merge: reviews.parent_asin to unique metadata.parent_asin, expected many-to-one. Require non-null keys, verify metadata uniqueness, report unmatched keys/reviews and pre/post counts, and reject row multiplication. Unmatched rate cannot be measured now. Listing-attribute comparison and comparable-subcategory grouping are blocked.

10. Reconcile changes - Done

1,000 input - 0 exact duplicates = 1,000 output. No other row exclusions and no merge. Rating sum 4,406 and helpful-vote sum 1,229 are unchanged. Rating counts for 1 through 5 stars: 72, 33, 47, 113, 735. Evidence: audit.before/after, rating_distribution, and source-preservation assertions.

11. Check calculations - Done

Average = sum of valid ratings / valid rating count; low-rating share = count <=2 / valid rating count. Both denominators are 1,000. Helpful total uses 1,000 valid votes. Missing values would be excluded from their metric only; missing parent keys would remain in an explicit null group. All 829 groups include counts/denominators; their row counts, rating sums and vote sums reconcile to overall totals. Evidence: product_metrics.json and clean.py assertions.

12. Verify five records - Done

Five expectations were written in expected_checks.json before the first cleaning run, after inspecting raw records. All five match actual output. Original, expected, actual and reasons appear on the following pages and in output/record_checks.json.

13. Test reproducibility - Done

Run: python3 jenny_guo/clean.py. A fresh Python 3.12.14 process wrote to a separate directory; all six data/audit output files matched SHA-256 byte-for-byte. The notebook code cells also executed sequentially in fresh Python; interactive Jupyter execution was not tested. Evidence: reproducibility.json. Cleaning requires only the standard library.

14. Document limitations - Done

Metadata join remains unresolved. Unknown sampling prevents population inference; the full upstream category was not checked. Subcategories and listing histories are unavailable. Ratings and complaints do not prove listing defects: quality, delivery and usage issues need separate interpretation. This item is Done because these limitations are documented, not because they are resolved.

Cleaning decision log

Rules and expectations were saved before applying transformations. See cleaning_rules.md for the full rule log; the following includes the actual verification results.

Source and IDs

Rule: Preserve source file and original values; add source_row. Keep all IDs as strings. Reason: Traceability and variant/parent distinctions must survive cleaning. Check: SHA-256 unchanged; every raw field matches; 389 variant differences preserved.

Ratings and dates

Rule: Accept integral ratings 1..5; convert Unix milliseconds to UTC. Invalid cleaned values become null and are flagged. Reason: Prevent invalid scales, time-unit mistakes and invented values. Check: 0 conversion failures; fixed UTC expectation passes; every date roundtrips to original milliseconds.

Review text

Rule: Replace known break tags, decode entities once and normalize whitespace. Reason: Improve readability without deleting wording or negation. Check: 314 bodies and 13 titles changed; full row 4 expectation matches.

Duplicates

Rule: Remove exact duplicate source records only; keep nonidentical repeated IDs. Reason: A listing can legitimately have multiple customer reviews. Check: 0 exact duplicates, 0 composite collisions; all 1,000 rows retained.

Missingness and images

Rule: Do not impute; treat empty image lists as no images. Reason: Missing evidence must not become invented evidence. Check: All per-column missing counts remain 0; 957 empty image lists retained.

Votes and purchase status

Rule: Retain valid large vote counts and unverified purchases. Reason: Unusual does not mean invalid; false is not null. Check: Row 92 retains 26 votes; row 60 retains false; total votes stays 1,229.

Merge and calculations

Rule: Defer missing metadata join; use explicit metric denominators. Reason: Avoid false unmatched statistics, row multiplication or unsupported mismatch scores. Check: No merge claimed; all 829 product groups reconcile to overall counts and sums.

Five record checks

The literal expected values below were recorded before clean.py was first executed. source_row is the original one-based file line. All five cases are retained in the 50-row sample.

Check 1 | source row 1 | timestamp_utc

Reason: 1519317108692 Unix milliseconds must retain the .692 seconds and use UTC.

Original: 1519317108692

Expected: "2018-02-22T16:31:48.692Z"

Actual: "2018-02-22T16:31:48.692Z"

Result: PASS

Check 2 | source row 4 | text_clean

Reason: Replace two HTML breaks by whitespace; keep wording, negation and punctuation.

Original: "I wasn't sure whether these were worth it or not, given the cost compared to the original branded filters.

I can happily report that these are a great value and work every bit as good as the original. If you are on the fence worrying whether these are worth it- I can assure you they are."

Expected: "I wasn't sure whether these were worth it or not, given the cost compared to the original branded filters. I can happily report that these are a great value and work every bit as good as the original. If you are on the fence worrying whether these are worth it- I can assure you they are."

Actual: "I wasn't sure whether these were worth it or not, given the cost compared to the original branded filters. I can happily report that these are a great value and work every bit as good as the original. If you are on the fence worrying whether these are worth it- I can assure you they are."

Result: PASS

Check 3 | source row 2 | parent_asin

Reason: Keep the parent distinct from variant ASIN B07DD2DMXB; do not replace it with asin.

Original: "B07DD37QPZ"

Expected: "B07DD37QPZ"

Actual: "B07DD37QPZ"

Result: PASS

Record checks and reproducibility

Check 4 | source row 60 | verified_purchase_clean

Reason: False is a valid unverified purchase, not missing data and not grounds for removal.

Original: false

Expected: false

Actual: false

Result: PASS

Check 5 | source row 92 | helpful_vote_clean

Reason: 26 helpful votes is a valid nonnegative count; retain it despite being unusually high in this sample.

Original: 26

Expected: 26

Actual: 26

Result: PASS

Verification of a Codex suggestion

Codex suggested explicit Unix-millisecond conversion to UTC. The automated verification compared source row 1 (1519317108692) with the prespecified value 2018-02-22T16:31:48.692Z and roundtripped all 1,000 converted timestamps to the original integers. This detects unit, timezone and precision errors. Raw values remain intact. Zitong Guo should review this evidence before submission; no personal student verification is claimed here.

Reproduction and included sample

Run from repository root: python3 jenny_guo/clean.py
Independent run: python3 jenny_guo/clean.py --output /tmp/jenny-independent-run
The six expected files are cleaned_reviews.jsonl, cleaned_sample_50.jsonl, audit.json, record_checks.json, excluded_rows.json and product_metrics.json under jenny_guo/output/. All matched in the independent run.

The 50-row sample contains source rows 1-48, 60 and 92: the five prespecified cases plus the earliest remaining rows. Selection emphasizes auditability and difficult cases and is not statistically representative. Original text remains alongside cleaned text.

Notebook: dataset/proj_data_clean_jenny_guo.ipynb. README and requirements: jenny_guo/. PDF rebuilding requires ReportLab 4.4.9: python3 jenny_guo/build_report.py. Instructor access and Gradescope submission must be confirmed separately.